



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
"ANTONIO JOSÉ DE SUCRE"
VICERRECTORADO "LUIS CABALLERO MEJÍAS"
GUARENAS

Exp: 2020200057 Apellido y Nombre: Guillermo Cardenas.

Exp: 2019200185 Apellido y Nombre: Juan Suarez.

Fecha: 2/10/2025

Ejercicio 1.

Se requiere del diseño de un sistema basado en búsquedas, aplicado a su proyecto de Inteligencia Artificial. Se desea que el sistema (sea juego, o cualquier sistema inteligente, según sea el caso del proyecto que usted esté realizando en la asignatura) alcance las destrezas de la máquina y las supere. Para ello, se espera que Usted seleccione el criterio de búsqueda y el algoritmo que permita al usuario:

1. Conocer la estrategia del sistema.
2. Hacer movimientos inesperados que permitan interactuar con el usuario.
3. Reaccionar a los desconocimientos de estrategias o movimientos del usuario.

Para resolver este escenario se espera que Usted diseñe un sistema inteligente basado en búsquedas, según el algoritmo de su preferencia, donde deberá indicar:

a) FORMALIZACIÓN (Puntos 5):

- **Estado (posición) inicial:** El sistema comienza en un estado donde el usuario ingresa una letra del abecedario la cual se va a concatenar y será buscada en la base de datos.
- **Operadores (movimientos):** Los movimientos se pueden enumerar de la siguiente manera:
 - 1) Leer la letra ingresada por el usuario.
 - 2) Buscar la imagen correspondiente a la letra en el directorio de imágenes.
 - 3) Mostrar la imagen encontrada.
- **Dirección del proceso de búsqueda:** Desde la entrada del usuario hasta encontrar la imagen correspondiente se realiza una búsqueda por profundidad a través de la base de datos.



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
"ANTONIO JOSÉ DE SUCRE"
VICERRECTORADO "LUIS CABALLERO MEJÍAS"
GUARENAS

- **Prueba de finalización:** El sistema finaliza una iteración del proceso cuando muestra la imagen correspondiente a la letra ingresada o indica que la imagen no se encontró.
- **Función de utilidad:** Esta se mide a través de la evaluación del éxito de la búsqueda, es decir, si se encuentra y muestra la imagen correcta, se considera un éxito.

b) APROXIMACIÓN PRÁCTICA (5 puntos):

1. Tipo de Búsqueda:

Búsqueda por Anchura (Breadth-First Search - BFS), esto debido a que BFS asegura que todos los estados posibles se exploren nivel por nivel, garantizando que se encuentre la solución si existe, además, encuentra la solución más cercana al estado inicial en términos de niveles de profundidad y es fácil de implementar y entender, utilizando una estructura de datos simple (una cola) para gestionar la exploración de nodos, evitando caer en ciclos infinitos .

2. Algoritmo a emplear en el lenguaje de programación python con la librería TensorFlow:

El código es el siguiente:

En esta parte se implementan las librerías necesarias para llevar a cabo el funcionamiento del proceso:

```
import os # Importa el módulo os para interactuar con el sistema de archivos
from PIL import Image # Importa la clase Image del módulo PIL para trabajar con imágenes
import matplotlib.pyplot as plt # Importa matplotlib para mostrar las imágenes
from collections import deque # Importa deque para realizar la búsqueda por anchura
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

Se define la ruta donde se encuentran ubicados los archivos y se almacenan en una variable:

```
# Ruta de la carpeta que contiene las imágenes
directorio_imagenes = r'C:\Users\asal\OneDrive\Documentos\Universidad\In
```

Se escribe la función encarga de realizar la búsqueda por anchura, con todas sus características:

```
# Función para realizar la búsqueda por anchura
def busqueda_anchura(letra):
    cola = deque([directorio_imagenes]) # Inicializa la cola con el directorio de imágenes

    while cola:
        directorio_actual = cola.popleft() # Obtiene y elimina el primer elemento de la cola
        for entrada in os.listdir(directorio_actual): # Itera sobre las entradas en el directorio actual
            ruta = os.path.join(directorio_actual, entrada) # Construye la ruta completa de la entrada
            if os.path.isdir(ruta): # Si la entrada es un directorio, la añade a la cola
                cola.append(ruta)
            elif os.path.isfile(ruta) and entrada.lower().startswith(letra.lower()) and entrada.endswith('.jpg'):
                # Si la entrada es un archivo y el nombre del archivo empieza con la letra dada y termina con '.jpg'
                return Image.open(ruta) # Abre y retorna la imagen

    return None # Retorna None si no se encuentra la imagen
```

Se crea la siguiente función encargada de mostrar las imágenes:

```
# Función para mostrar la imagen
def mostrar_imagen(imagen):
    if imagen is not None:
        plt.imshow(imagen) # Muestra la imagen usando matplotlib
        plt.axis('off') # Oculta los ejes
        plt.show() # Muestra la ventana con la imagen
    else:
        print("Imagen no encontrada.") # Imprime un mensaje si no se encuentra la imagen
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

Se programa la función que realiza la solicitud al usuario de la letra:

```
def main():  
    while True:  
        letra = input("Introduce una letra del abecedario o presiona el punto: ").upper() # Solicita una letra a  
        if letra == '.':  
            break # Si el usuario introduce '.', termina el programa  
        elif len(letra) != 1 or not letra.isalpha():  
            print("Por favor, introduce una sola letra válida.") # Verifica que la entrada sea una letra válida  
            continue  
  
        imagen = busqueda_anchura(letra) # Realiza la búsqueda de la imagen correspondiente a la letra  
        mostrar_imagen(imagen) # Muestra la imagen encontrada
```

Iniciador del script:

```
# Verifica si el script es ejecutado directamente y llama a la función principal  
if __name__ == "__main__":  
    main()
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
"ANTONIO JOSÉ DE SUCRE"
VICERRECTORADO "LUIS CABALLERO MEJÍAS"
GUARENAS

Ejercicio 2.

Basado en el ejercicio 1, diseñe un sistema Experto que cumpla con las siguientes características:

1. Permite buscar los conocimientos apropiados y a partir de éstos producir nuevos conocimientos
2. Posee capacidad de representación simbólica del conocimiento y razonamiento.
3. Referencia a un dominio de conocimiento técnico y altamente especializado.
4. Capacidad de proceder a realizar la búsqueda de soluciones.
5. Está obligado a explicar sus razonamientos, preguntas y conclusiones.
6. Puede usar datos erróneos, reglas inciertas y manejo de incertidumbre.
7. Emplea generalmente interfaz de lenguaje natural.
8. Interacción con el humano o con el medio que controlan.

Para elaborar este sistema Usted deberá indicar:

A. ARQUITECTURA DEL SISTEMA EXPERTO (2 PUNTOS):

- **Interfaz de Usuario :** Permite la interacción con el programa mediante captura de pantalla en tiempo real para la detección de lenguaje de señas.
- **Base de Conocimientos (BK):** Incluye datos de entrenamiento y prueba de letras del lenguaje de señas, almacenados en archivos CSV, y reglas para la correspondencia entre letras y sus imágenes.
- **Motor de Inferencia:** Red neuronal convolucional (CNN) entrenada con TensorFlow para clasificar las letras en lenguaje de señas.
- **Módulo de Explicaciones:** Proporciona predicciones y niveles de confianza sobre las letras identificadas.

Asignatura: Inteligencia Artificial	Tema: Prueba Escrita N° 2 (25%)	Prof(a): Mónica Tahan	Pag. 5 De 12
-------------------------------------	---------------------------------	-----------------------	--------------



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

B. PATRONES DE CONOCIMIENTO INICIAL (3 PUNTOS):

- **Hechos Iniciales:** En este caso se tienen los datos de entrenamiento y prueba de letras en lenguaje de señas almacenados en archivos CSV, los provienen de una serie de imágenes que las cuales fueron procesadas para llevarlas a una dimensión de 28x28 pixeles y su escala de grises, mediante la cual se determinó, el valor del 0-255 en escala de negros, que permite identificar las diferentes características de cada letra. Concluyen en un archivo CSV de 786 columnas rellanas de esta información.
- **Reglas:** Se tienen las reglas de correspondencia que relacionan las letras del abecedario con sus imágenes en lenguaje de señas, además de las reglas para preprocesar las imágenes y convertirlas al formato adecuado para la CNN.

C. HARDWARE Y SOFTWARE (ALGORITMO EMPLEANDO EL LENGUAJE DE PROGRAMACIÓN PYTHON CON LA LIBRERÍA TENSORFLOW) (5 PUNTOS):

- **Hardware:** Se tiene un sistema de captura de imágenes en tiempo real, como puede ser una cámara o la captura de pantalla, a su vez, se tiene la capacidad de procesar imágenes utilizando MediaPipe y TensorFlow, y suficiente capacidad de almacenamiento para manejar los datos de entrenamiento y prueba, además del modelo generado durante esta fase.
- **Software:** Se programo mediante el lenguaje de programación Python, usando VisualStudioCode como entorno de programación. A este lenguaje se le añadieron las siguientes librerías, necesarias para llevar a cabo nuestro proyecto: TensorFlow, Keras, OpenCV, MediaPipe, MSS, NumPy, Pandas.
- **Código Implementado:** El código se divide en dos partes, 1ro la parte de entrenamiento de la red CNN, y luego el programa con entorno que ejecuta esta red.



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

1ra parte del código, encargada de extraer los archivos de entrenamiento y prueba, definir la red CNN, su pasas y crear el modelo:

```
8 import tensorflow as tf
9 from tensorflow.keras import layers, models
10 from tensorflow.keras.utils import to_categorical
11 import numpy as np
12 import pandas as pd
13 from sklearn.model_selection import train_test_split
14
15 # Cargar el dataset desde archivos CSV
16 train_df = pd.read_csv('Datos_csv\sign_mnist_train.csv') # Cargar datos de entrenamiento
17 test_df = pd.read_csv('Datos_csv\sign_mnist_test.csv') # Cargar datos de prueba
18
19 # Función para preprocesar los datos
20 def preprocess_data(df):
21     # Extraer las etiquetas (labels) de la columna 'label'
22     labels = df['label'].values
23     # Extraer las imágenes (todas las columnas excepto 'label') y convertirlas a un array de numpy
24     images = df.drop(columns=['label']).values
25     # Reformar las imágenes a un formato de (28, 28, 1) para que sean compatibles con una CNN
26     images = images.reshape(-1, 28, 28, 1)
27     # Normalizar los valores de píxeles al rango [0, 1]
28     images = images / 255.0
29     return images, labels
30
31 # Preprocesar los datos de entrenamiento y prueba
32 x_train, y_train = preprocess_data(train_df) # Preprocesar datos de entrenamiento
33 x_test, y_test = preprocess_data(test_df) # Preprocesar datos de prueba
34
35 # Convertir las etiquetas a one-hot encoding
36 num_classes = 25 # Número de clases (24 letras del lenguaje de señas, excluyendo J y Z)
37 y_train = to_categorical(y_train, num_classes) # Convertir etiquetas de entrenamiento
38 y_test = to_categorical(y_test, num_classes) # Convertir etiquetas de prueba
39
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

```
# Definir el modelo de red neuronal convolucional (CNN)
model = models.Sequential([
    # Primera capa convolucional con 32 filtros de 3x3 y función de activación ReLU
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    # Capa de MaxPooling para reducir la dimensionalidad
    layers.MaxPooling2D((2, 2)),
    # Segunda capa convolucional con 64 filtros de 3x3 y función de activación ReLU
    layers.Conv2D(64, (3, 3), activation='relu'),
    # Otra capa de MaxPooling
    layers.MaxPooling2D((2, 2)),
    # Tercera capa convolucional con 128 filtros de 3x3 y función de activación ReLU
    layers.Conv2D(128, (3, 3), activation='relu'),
    # Última capa de MaxPooling
    layers.MaxPooling2D((2, 2)),
    # Capa de Dropout para evitar el sobreajuste (desactiva el 50% de las neuronas aleatoriamente)
    tf.keras.layers.Dropout(0.5),
    # Aplanar la salida para conectarla a una capa densa
    layers.Flatten(),
    # Capa densa (fully connected) con 128 neuronas y función de activación ReLU
    layers.Dense(128, activation='relu'),
    # Capa de salida con 'num_classes' neuronas (una por cada clase) y activación softmax
    layers.Dense(num_classes, activation='softmax') # 24 clases (A-Y, excluyendo J y Z)
])

# Compilar el modelo
model.compile(optimizer='adam', # Usar el optimizador Adam
              loss='categorical_crossentropy', # Función de pérdida para clasificación multiclase
              metrics=['accuracy']) # Métrica a monitorear: precisión

# Entrenar el modelo
history = model.fit(X_train, y_train, # Datos de entrenamiento
                   epochs=10, # Número de épocas
                   batch_size=32, # Tamaño del lote
                   validation_data=(X_test, y_test)) # Datos de validación

# Evaluar el modelo en el conjunto de prueba
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f'\nPrecisión en el conjunto de prueba: {test_acc:.4f}') # Mostrar la precisión en prueba

# Guardar el modelo entrenado en un archivo
model.save('Modelo_lenguaje_senas_5.keras') # Guardar el modelo en formato .keras
```




U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
"ANTONIO JOSÉ DE SUCRE"
VICERRECTORADO "LUIS CABALLERO MEJÍAS"
GUARENAS

2da parte del código, encargada de correr el modelo entrenado y proporcionar la herramienta de captura:

```
6
7  import tensorflow as tf
8  import numpy as np
9  import cv2
10 import mediapipe as mp
11 from mss import mss
12
13 # Cargar el modelo entrenado
14 model = tf.keras.models.load_model("Modelo_lenguaje_senas_5.keras")
15
16 # Inicializar MediaPipe Hands
17 mp_hands = mp.solutions.hands
18 hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7)
19
20 # Etiquetas de las clases (excluyendo J y Z)
21 class_labels = "ABCDEFGHIKLMNOPQRSTUVWXYZ"
22
23 # Función para preprocesar la imagen
24 def preprocess_image(image):
25     image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Convertir a escala de grises
26     image = cv2.resize(image, (28, 28)) # Redimensionar a 28x28
27     image = image / 255.0 # Normalizar
28     image = image.reshape(1, 28, 28, 1) # Reformar para el modelo
29     return image
30
31 # Función para predecir la clase de la imagen
32 def predict_image(image):
33     prediction = model.predict([image])
34     predicted_class = np.argmax(prediction)
35     confidence = np.max(prediction)
36     return predicted_class, confidence
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
"ANTONIO JOSÉ DE SUCRE"
VICERRECTORADO "LUIS CABALLERO MEJÍAS"
GUARENAS

```
# Configurar la captura de pantalla con mss
with mss() as sct:
    # Definir el área de captura (600x600 píxeles centrados)
    screen_width = sct.monitors[1]["width"]
    screen_height = sct.monitors[1]["height"]
    monitor = {
        "top": (screen_height - 600) // 2, # Centrar verticalmente
        "left": (screen_width - 600) // 2, # Centrar horizontalmente
        "width": 600,
        "height": 600
    }

    print("Captura de pantalla activada. Presiona 'q' para salir.")
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

```
while True:
    # Capturar el recuadro de la pantalla
    screenshot = np.array(sct.grab(monitor))

    # Convertir la imagen a RGB (MediaPipe requiere imágenes en RGB)
    rgb_frame = cv2.cvtColor(screenshot, cv2.COLOR_BGRA2RGB)

    # Procesar el frame con MediaPipe Hands
    results = hands.process(rgb_frame)

    # Si se detecta una mano, dibujar un recuadro azul alrededor de ella
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            # Obtener las coordenadas de los landmarks de la mano
            h, w, _ = screenshot.shape
            x_min, y_min, x_max, y_max = w, h, 0, 0
            for landmark in hand_landmarks.landmark:
                x, y = int(landmark.x * w), int(landmark.y * h)
                if x < x_min:
                    x_min = x
                if x > x_max:
                    x_max = x
                if y < y_min:
                    y_min = y
                if y > y_max:
                    y_max = y

            # Dibujar un recuadro azul alrededor de la mano
            cv2.rectangle(screenshot, (x_min, y_min), (x_max, y_max), (255, 0, 0), 2)
```



U
N
E
X
P
O

REPÚBLICA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL POLITÉCNICA
“ANTONIO JOSÉ DE SUCRE”
VICERRECTORADO “LUIS CABALLERO MEJÍAS”
GUARENAS

```
# Recortar la región de la mano
hand_region = screenshot[y_min:y_max, x_min:x_max]

# Si la región de la mano es válida, preprocesarla y predecir
if hand_region.size != 0:
    processed_image = preprocess_image(hand_region)
    predicted_class, confidence = predict_image(processed_image)

    # Verificar que predicted_class esté dentro del rango válido
    if 0 <= predicted_class < len(class_labels):
        label = f"Predicción: {class_labels[predicted_class]} (Confianza: {confidence:.2f})"
    else:
        label = "Predicción: Desconocido"

    # Mostrar la predicción en el frame
    cv2.putText(screenshot, label, (x_min, y_min - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 0, 0), 2)

# Mostrar el frame en una ventana
cv2.imshow('Captura de Pantalla - Detección de Manos', screenshot)

# Salir si se presiona la tecla 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cv2.destroyAllWindows()
```